

Cidades ESG Inteligentes

Documentação técnica de DevOps
Pipeline CI/CD · Docker · Staging e Produção

5

jobs no pipeline

11

testes automatizados

2

ambientes de deploy

100%

execução com sucesso

INTEGRANTES · GRUPO 14

Fase: Navegando pelo mundo DevOps

- Amanda do Nascimento Diacís
RM565236
- Ariane Hemerich
RM561659
- Ariel de Souza Lima
RM561994
- Danilo Lima Cruz
RM564828
- Paulo Augusto Siqueira
RM563083

github.com/dlcruzz/cidades-esg-inteligentes

Java 17 · Spring Boot 3.3 · GitHub Actions

Sumário

- 01 Visão geral do projeto
- 02 Arquitetura da solução
- 03 Pipeline CI/CD
- 04 Docker e orquestração
- 05 Evidências do pipeline
- 06 Evidências de staging e produção
- 07 Desafios encontrados e soluções
- 08 Checklist de entrega

Execução de referência do pipeline: github.com/dlcruzz/cidades-esg-inteligentes/actions/runs/37162471208
Imagem publicada: ghcr.io/dlcruzz/cidades-esg-inteligentes:c1fc108... e :latest
Resultado: 5/5 jobs com sucesso · 11/11 testes aprovados · deploy validado em staging e produção

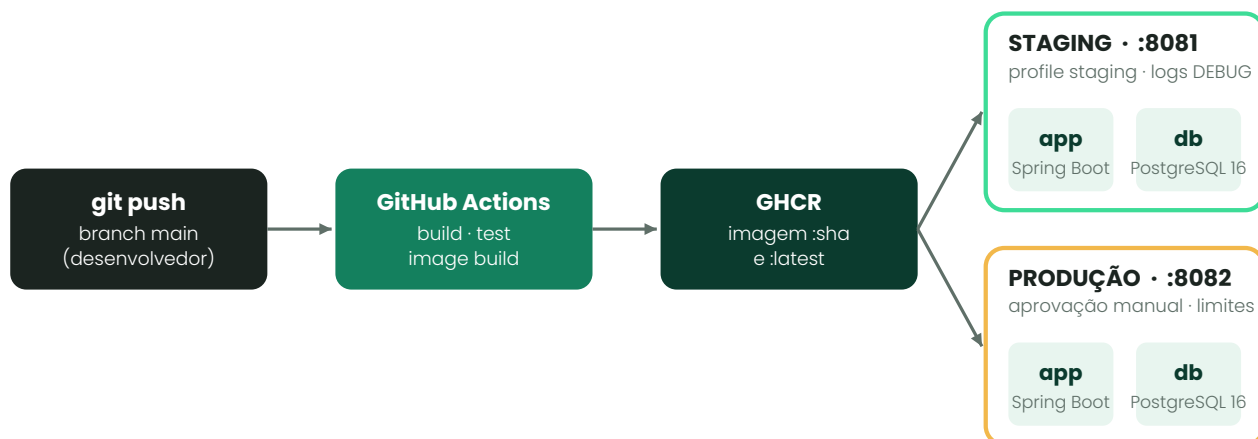
1 Visão geral do projeto

A **Cidades ESG Inteligentes** é uma API REST em Java Spring Boot para cadastrar cidades e acompanhar seus indicadores **Ambiental**, **Social** e de **Governança** (0 a 100), com cálculo automático do score ESG. Neste desafio, a aplicação ganhou um ciclo de vida completo de DevOps: a cada push na branch *main*, o código é compilado, testado, empacotado como imagem Docker e implantado automaticamente em **staging** e, após aprovação, em **produção**, com validação automática em cada ambiente.

Método	Rota	Descrição
GET	/	Status da API e ambiente ativo (staging / production)
GET	/api/cidades	Lista cidades — filtros ?uf= e ?nome=
GET	/api/cidades/{id}	Busca por id (404 se não existir)
POST · PUT · DELETE	/api/cidades[/]{id}	Cria, atualiza e remove cidades
GET	/actuator/health	Health check usado por Docker, Kubernetes e pipeline

2 Arquitetura da solução

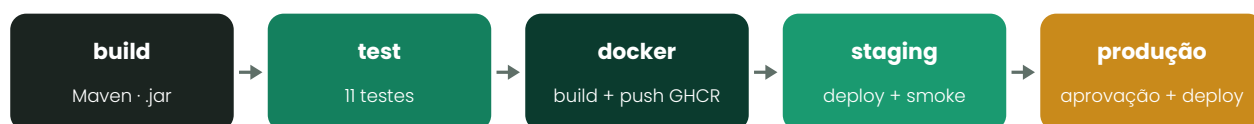
O fluxo começa no push do desenvolvedor e termina com a mesma imagem rodando em dois ambientes isolados. Cada ambiente é um projeto Docker Compose separado, com rede, volume e porta próprios.



Componente	Detalhe
Repositório	github.com/dlcruzz/cidades-esg-inteligentes
CI/CD	GitHub Actions + GitHub Environments (staging / production)
Registro de imagens	GitHub Container Registry (ghcr.io)
Orquestração	Docker Compose (app + db, rede, volume, variáveis) · manifests Kubernetes alternativos
Banco de dados	PostgreSQL 16 nos ambientes · H2 em memória nos testes

3 Pipeline CI/CD

Ferramenta: GitHub Actions — arquivo [.github/workflows/ci-cd.yml](#).



Gatilhos

- **push na main** — pipeline completo, até produção.
- **pull_request para main** — só build e testes, como gate de qualidade antes do merge.
- **workflow_dispatch** — execução manual pela aba Actions.

Etapas e lógica

Job	O que faz
build	Configura JDK 17, executa <code>mvn clean package -DskipTests</code> e publica o .jar como artefato.
test	Roda 11 testes (JUnit 5, Mockito, MockMvc) com o profile <code>test</code> em H2. Publica o relatório Surefire e um resumo na página da execução.
docker-build-push	Constrói a imagem pelo Dockerfile e publica no GHCR com as tags do SHA do commit e <code>latest</code> .
deploy-staging	Environment <i>staging</i> . Baixa a imagem e roda <code>scripts/deploy.sh staging</code> : Compose sobe API + Postgres e o smoke test valida tudo.
deploy-production	Só roda se staging passou. Environment <i>production</i> com aprovação manual . Roda <code>scripts/deploy.sh production</code> com o override de produção.

Staging × produção

	Staging	Produção
Disparo	Automático após os testes	Após aprovação manual
Profile Spring	staging	production
Porta	8081	8082
Logs	DEBUG + SQL	INFO, sem SQL
Health check	Detalhado (banco, disco)	Somente status
Banco	Porta exposta para depuração	Somente rede interna
Recursos	Sem limites	1 CPU / 512 MB · restart: always

Smoke test

O script `scripts/smoke-test.sh` espera o `/actuator/health` ficar UP, confere o ambiente ativo, cria uma cidade (POST), consulta por UF (GET) e exige 404 para um id inexistente. Qualquer falha interrompe o pipeline e bloqueia a promoção para produção.

Build once, deploy many: staging e produção recebem exatamente a mesma imagem (tag do commit). Só a configuração muda — o que foi testado em staging é o que vai para produção.

4 Docker e orquestração

Arquitetura da imagem

Build **multi-stage**: o primeiro estágio usa *maven:3.9-eclipse-temurin-17* para compilar; o estágio final parte de *eclipse-temurin:17-jre-jammy* e recebe só o *.jar*. Resultado: imagem menor, sem Maven/JDK e com menos superfície de ataque.

```
# =====
# Dockerfile - Cidades ESG Inteligentes
# Estratégia: build multi-stage para manter a imagem final
# enxuta (sem Maven/JDK completo) e mais segura (usuário
# não-root, apenas JRE em runtime).
# =====

# ----- STAGE 1: build -----
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copia apenas o pom.xml primeiro para aproveitar o cache de camadas
# do Docker: dependências só são baixadas de novo se o pom mudar.
COPY pom.xml .
RUN mvn -B dependency:go-offline

# Copia o restante do código-fonte e gera o artefato .jar
COPY src ./src
RUN mvn -B clean package -DskipTests

# ----- STAGE 2: runtime -----
FROM eclipse-temurin:17-jre-jammy AS runtime

# wget é usado apenas pelo HEALTHCHECK abaixo
RUN apt-get update && apt-get install -y --no-install-recommends wget \
    && rm -rf /var/lib/apt/lists/*

# Usuário não-root por segurança
RUN groupadd -r esg && useradd -r -g esg esgapp

WORKDIR /app

# Copia apenas o .jar gerado no estágio de build (imagem final não tem Maven/JDK)
COPY --from=build /app/target/cidades-esg-inteligentes.jar app.jar

RUN chown -R esgapp:esg /app
USER esgapp

EXPOSE 8080

# Healthcheck usado pelo docker-compose / orquestrador
HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \
    CMD wget --no-verbose --tries=1 --spider http://localhost:8080/actuator/health || exit 1

# MaxRAMPercentage: a JVM respeita o limite de memória do container
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Estratégias adotadas

- **Cache de dependências** — o *pom.xml* é copiado antes do código; mudar só o código não baixa tudo de novo.
- **Usuário não-root** (esgapp) executando a aplicação.
- **HEALTHCHECK** nativo consultando o Actuator.
- **-XX:MaxRAMPercentage=75** para a JVM respeitar o limite de memória do container.
- **.dockerignore** evita enviar *target/*, *.git/*, *docs/* e *.env* ao contexto de build.

Docker Compose

Recurso	Como foi usado
Serviços	app (Spring Boot) e db (postgres:16-alpine); a API só sobe com o banco saudável (depends_on: service_healthy)
Rede	esg-network (bridge), isolada por ambiente
Volume	esg-db-data — persistência dos dados do PostgreSQL
Variáveis	Arquivo .env (modelo .env.example): profile, porta, credenciais e imagem
Override	docker-compose.production.yml: profile production, limites, restart always, banco sem porta

Comandos utilizados

```
docker compose --env-file .env up -d --build    # ambiente local
docker compose ps                               # status dos containers
bash scripts/deploy.sh staging <imagem>         # usado pelo pipeline -> :8081
bash scripts/deploy.sh production <imagem>     # usado pelo pipeline -> :8082
bash scripts/destroy.sh staging --volumes       # derruba um ambiente
kubectl apply -f k8s/                           # alternativa Kubernetes
```

Imagem criada: ghcr.io/dlcruzz/cidades-esg-inteligentes — tags

c1fc10833b8231cdfcd466b149bf0384935b919f e latest. **Alternativa Kubernetes:** pasta k8s/ com Deployment (2 réplicas, probes no /actuator/health), StatefulSet do Postgres com PVC e ConfigMap/Secret.

5 Evidências do pipeline

Pipeline completo · 01-pipeline.png

The screenshot shows the GitHub Actions interface for the repository 'drcruzz / cidades-esg-inteligentes'. The selected workflow is 'Cidades ESG Inteligentes - CI/CD, Docker e deploy staging/producao #1'. The status is 'Success' with a total duration of '6m 32s' and 5 artifacts. The pipeline steps are: 'Build da aplicação' (25s), 'Testes automatizados' (18s), 'Build & Push da imagem Docker' (58s), 'Deploy em Staging' (40s), and 'Deploy em Produção' (40s). A 'Testes automatizados summary' section shows the following test results:

```
Resultado dos testes

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.418 s -- in com.esg.cidades.CidadesEsgApplicationTests
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.341 s -- in com.esg.cidades.controller.CidadeControllerTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.257 s -- in com.esg.cidades.service.CidadeServiceTest
```

Execução #1 — os 5 jobs concluídos com sucesso em 6m32s.

Testes automatizados · 02-testes.png

The screenshot shows the 'Testes automatizados summary' section, which displays the 'Resultado dos testes' (Test Results). The summary shows the following test results:

```
Resultado dos testes

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.418 s -- in com.esg.cidades.CidadesEsgApplicationTests
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.341 s -- in com.esg.cidades.controller.CidadeControllerTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.257 s -- in com.esg.cidades.service.CidadeServiceTest
```

11 testes executados, 0 falhas, 0 erros (resumo gerado pelo próprio pipeline).

Imagem no GitHub Container Registry · 03-imagem-ghcr.png

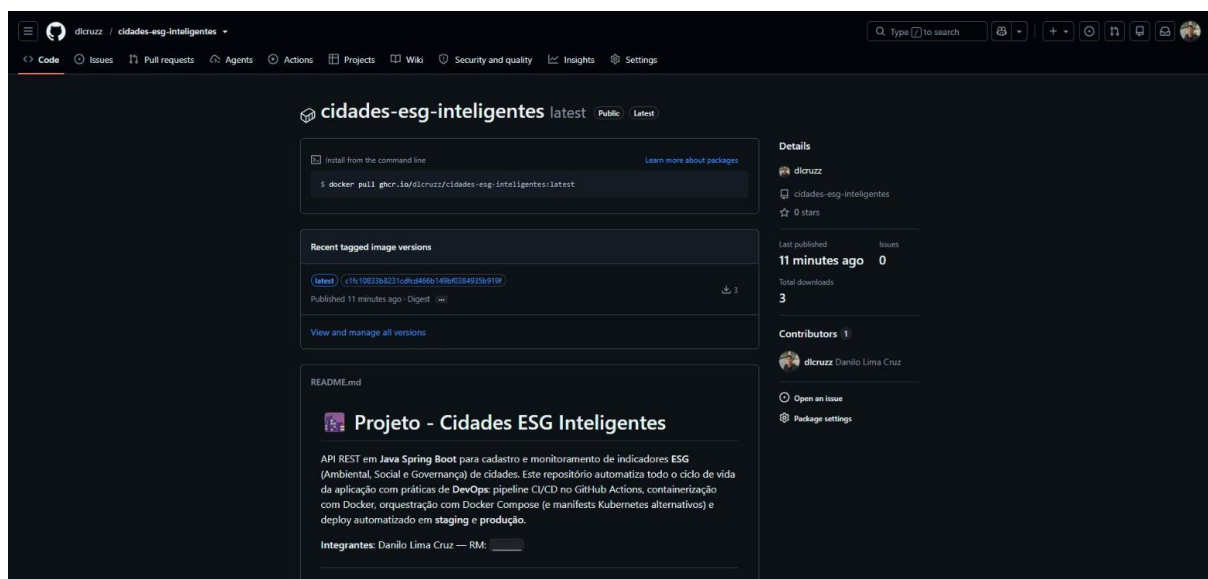
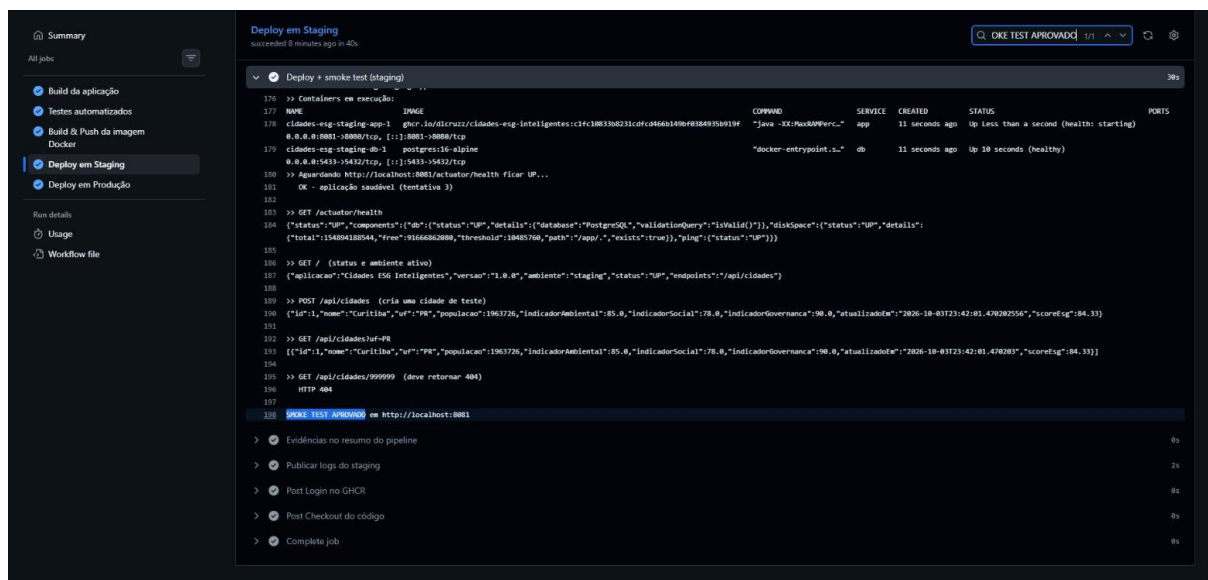


Imagem pública publicada com as tags do commit e latest.

6

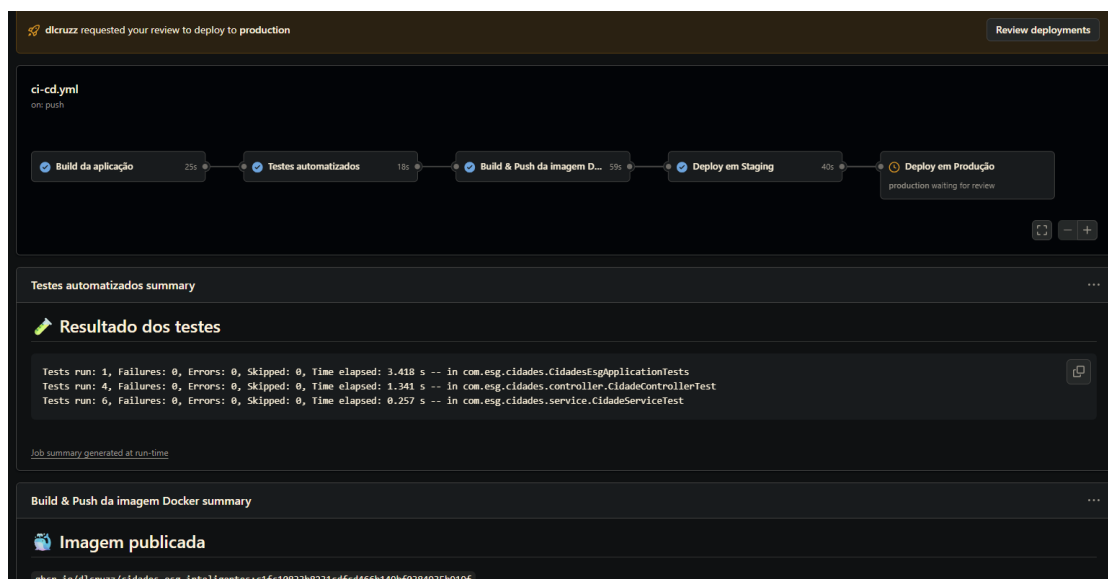
Evidências de staging e produção

Deploy em STAGING · 04-deploy-staging.png



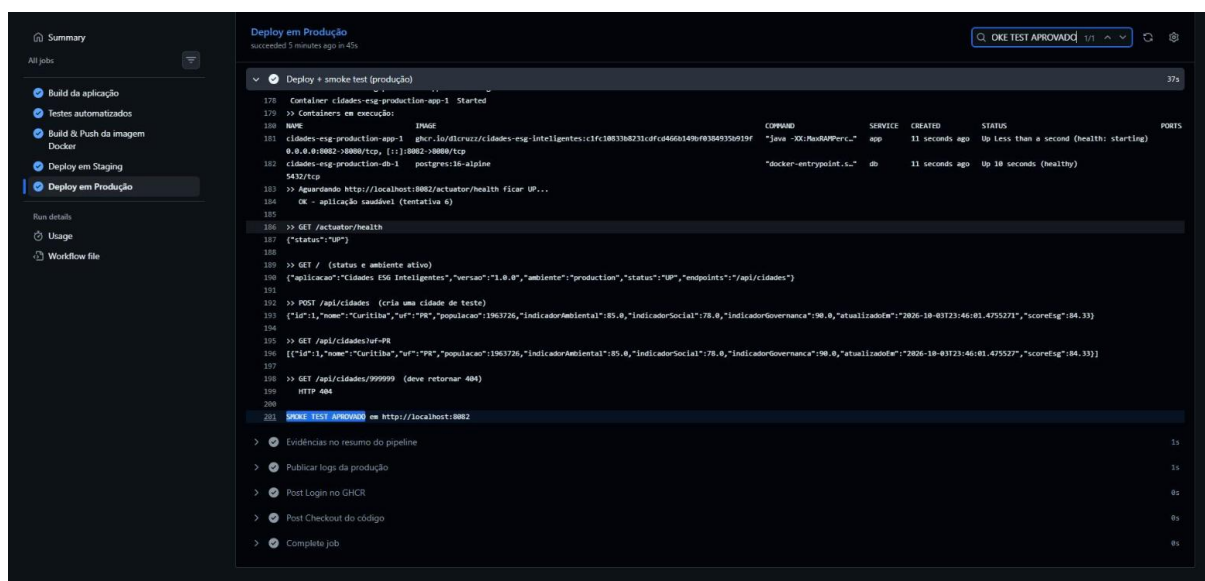
Containers app e db rodando, health UP, "ambiente": "staging", POST/GET funcionando, 404 validado e SMOKE TEST APROVADO.

Aprovação manual de produção · 05-aprovacao-producao.png



Pipeline aguardando revisão antes do deploy em production (GitHub Environments).

Deploy em PRODUÇÃO · 06-deploy-producao.png



"ambiente": "production", health sem detalhes (configuração de produção) e SMOKE TEST APROVADO.

7 Desafios encontrados e soluções

Desafio	Como foi resolvido
Testes no CI dependiam de um PostgreSQL rodando	Profile <i>test</i> com H2 em memória e dialeto H2 explícito.
Produção não subia em banco novo (ddl-auto=validate exige a tabela)	Produção passou a usar <i>update</i> : cria a tabela no primeiro deploy e nunca apaga dados.
Staging e produção não rodavam juntos (container_name fixo)	Cada ambiente virou um projeto Compose (-p) com rede, volume e porta próprios.
Job de produção usava a tag de imagem de um job fora do <i>needs</i> (valor vazio)	<i>needs</i> : [docker-build-push, deploy-staging] — produção recebe a mesma imagem de staging.
Deploy era apenas simulado com echo	Script <i>deploy.sh</i> real com Docker Compose + <i>smoke-test.sh</i> com requisições HTTP.
Falha de deploy escondida pelo / tee do log	Shell padrão do workflow com -eo pipefail.
GHCR rejeita nomes de imagem com maiúsculas	Nome da imagem convertido para minúsculas no pipeline.
Scripts .sh editados no Windows quebram no Linux (CRLF)	.gitattributes força final de linha LF em .sh, .yaml e Dockerfile.
Expor detalhes internos em produção	Sem <i>stacktrace</i> /mensagem nos erros, <i>health</i> sem detalhes e banco sem porta publicada.
Virtualização indisponível na máquina local para o Docker Desktop	Os ambientes foram executados e evidenciados no runner do GitHub Actions, com o mesmo Compose e scripts.

8 Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✓
Dockerfile funcional	✓
docker-compose.yml ou arquivos Kubernetes	✓
Pipeline com etapas de build, teste e deploy	✓
README.md com instruções e prints	✓
Documentação técnica com evidências (PDF ou PPT)	✓
Deploy realizado nos ambientes staging e produção	✓